

VendorVault Project Structure

Overview

Mobile app for Zimbabwe's informal grocery vendors - offline-first Flutter app with Firebase backend for inventory management, sales processing, and customer loyalty.

Key Requirements

- **Target**: Small informal grocery vendors in Zimbabwe
- **Platform**: Android-first (94% mobile penetration)
- **Connectivity**: Offline-first with intermittent sync
- **Users**: Low-tech literacy, minimal typing required
- **Cost**: Low deployment and operation costs

📱 FRONTEND (Flutter Mobile App)

Project Root Structure

```

```
vendorvault/
├── frontend/ # Flutter mobile app
│ ├── lib/
│ ├── pubspec.yaml
│ ├── android/
│ ├── ios/
│ └── test/
├── backend/ # Backend services (future)
├── shared/ # Shared models and utilities
└── docs/ # Documentation
```
```

Frontend Core Structure (`/frontend/lib/`)

```

```
frontend/lib/
├── main.dart
├── app.dart
└── core/
 ├── constants/
 │ ├── app_constants.dart
 │ ├── api_constants.dart
 │ └── ui_constants.dart
 ├── utils/
 │ ├── connectivity_helper.dart
 │ ├── storage_helper.dart
 │ ├── barcode_scanner.dart
 │ └── voice_input.dart
 ├── themes/
 │ ├── app_theme.dart
 │ └── colors.dart
 └── widgets/
 ├── common/
 │ ├── custom_button.dart
 │ ├── custom_text_field.dart
 │ ├── loading_indicator.dart
 │ └── offline_banner.dart
 ├── inventory/
 │ ├── stock_item_card.dart
 │ ├── low_stock_alert.dart
 │ └── expiry_warning.dart
 └── sales/
 ├── product_scanner.dart
 ├── checkout_item.dart
 └── payment_method.dart
```
```

```

├── loyalty/
│   ├── points_display.dart
│   ├── reward_card.dart
│   └── customer_info.dart
├── services/
│   ├── local_storage_service.dart
│   ├── sync_service.dart
│   ├── auth_service.dart
│   └── notification_service.dart
...

```

Frontend Feature Modules (`/frontend/lib/features/`)

Inventory Management (`/frontend/lib/features/inventory/`)

```

frontend/lib/features/inventory/
├── data/
│   ├── models/
│   │   ├── product_model.dart
│   │   ├── stock_item.dart
│   │   └── inventory_model.dart
│   ├── repositories/
│   │   ├── inventory_repository.dart
│   │   └── inventory_repository_impl.dart
│   └── datasources/
│       ├── local/
│       │   ├── inventory_local_datasource.dart
│       │   └── hive_adapters.dart
│       └── remote/
│           └── inventory_remote_datasource.dart
├── domain/
│   ├── entities/
│   │   ├── product.dart
│   │   └── stock_item.dart
│   ├── usecases/
│   │   ├── add_stock_item.dart
│   │   ├── update_stock.dart
│   │   ├── get_low_stock_items.dart
│   │   └── perform_stock_count.dart
│   └── repositories/
│       └── inventory_repository.dart
├── presentation/
│   ├── pages/
│   │   ├── inventory_home.dart
│   │   ├── add_stock_page.dart
│   │   ├── stock_count_page.dart
│   │   └── low_stock_alerts_page.dart
│   ├── widgets/
│   │   ├── stock_form.dart
│   │   ├── stock_list.dart
│   │   ├── voice_input_widget.dart
│   │   └── low_stock_alerts.dart
│   └── providers/
│       └── inventory_provider.dart
...

```

Sales Processing (`/frontend/lib/features/sales/`)

```

frontend/lib/features/sales/
├── data/
│   ├── models/
│   │   ├── sale_model.dart
│   │   ├── sale_item.dart
│   │   └── receipt_model.dart

```

```

├── cart_item.dart
├── repositories/
│   ├── sales_repository.dart
│   └── sales_repository_impl.dart
├── datasources/
│   ├── local/
│   │   ├── sales_local_datasource.dart
│   │   └── cart_local_datasource.dart
│   └── remote/
│       └── sales_remote_datasource.dart
├── domain/
│   ├── entities/
│   │   ├── sale.dart
│   │   ├── receipt.dart
│   │   └── cart_item.dart
│   ├── usecases/
│   │   ├── process_sale.dart
│   │   ├── scan_product.dart
│   │   ├── calculate_total.dart
│   │   └── generate_receipt.dart
│   └── repositories/
│       └── sales_repository.dart
├── presentation/
│   ├── pages/
│   │   ├── checkout_page.dart
│   │   ├── sales_history_page.dart
│   │   └── receipt_page.dart
│   ├── widgets/
│   │   ├── barcode_scanner_widget.dart
│   │   ├── checkout_summary.dart
│   │   ├── payment_options.dart
│   │   └── receipt_preview.dart
│   └── providers/
│       └── sales_provider.dart
...

```

Performance Analytics (`/frontend/lib/features/analytics/`)

```

frontend/lib/features/analytics/
├── data/
│   ├── models/
│   │   ├── sales_metrics.dart
│   │   ├── profit_analysis.dart
│   │   └── customer_insights.dart
│   ├── repositories/
│   │   ├── analytics_repository.dart
│   │   └── analytics_repository_impl.dart
│   └── datasources/
│       ├── local/
│       │   └── analytics_local_datasource.dart
│       └── remote/
│           └── analytics_remote_datasource.dart
├── domain/
│   ├── entities/
│   │   ├── daily_sales.dart
│   │   └── profitability_report.dart
│   ├── usecases/
│   │   ├── get_sales_trends.dart
│   │   ├── calculate_profit_margins.dart
│   │   └── get_top_products.dart
│   └── repositories/
│       └── analytics_repository.dart
├── presentation/
│   └── pages/

```



```

├── firestore/                # Firestore database rules
├── storage/                 # Firebase Storage rules
├── hosting/                  # Firebase Hosting config
└──

```

Firebase Functions (`/backend/functions/`)

```

backend/functions/
├── src/
│   ├── index.ts
│   ├── auth/
│   │   ├── userAuth.ts
│   │   └── tokenValidation.ts
│   ├── inventory/
│   │   ├── syncInventory.ts
│   │   └── validateStock.ts
│   ├── sales/
│   │   ├── processSale.ts
│   │   ├── calculateTotals.ts
│   │   └── generateReceipt.ts
│   ├── loyalty/
│   │   ├── calculatePoints.ts
│   │   ├── applyRewards.ts
│   │   └── sendSMS.ts
│   └── analytics/
│       ├── generateReports.ts
│       └── calculateMetrics.ts
├── package.json
├── tsconfig.json
└──

```

Firestore Database Structure (`/backend/firestore/`)

```

backend/firestore/
├── collections/
│   ├── vendors/
│   │   ├── {vendorId}/
│   │   │   ├── profile/
│   │   │   ├── settings/
│   │   │   └── subscriptions/
│   │   └──
│   ├── products/
│   │   ├── {productId}/
│   │   │   ├── basic_info/
│   │   │   ├── pricing/
│   │   │   └── barcode_data/
│   │   └──
│   ├── inventory/
│   │   ├── {vendorId}/
│   │   │   ├── stock_items/
│   │   │   ├── stock_counts/
│   │   │   └── low_stock_alerts/
│   │   └──
│   ├── sales/
│   │   ├── {vendorId}/
│   │   │   ├── transactions/
│   │   │   ├── receipts/
│   │   │   └── daily_summaries/
│   │   └──
│   ├── customers/
│   │   ├── {customerId}/
│   │   │   ├── profile/
│   │   │   ├── loyalty_account/
│   │   │   ├── points_history/
│   │   │   └── purchase_history/
│   │   └──
│   └── analytics/
│       ├── {vendorId}/
│       │   └── daily_metrics/
│       └──
└──

```

```
├── weekly_reports/
├── profit_analysis/
├── indexes/
│   └── composite_indexes.json
├── security_rules/
│   └── firestore.rules
└── ...
```

📁 SHARED COMPONENTS

Shared Models and Utilities (`/shared/`)

```
shared/
├── models/                                # Shared data models
│   ├── base_model.dart
│   ├── user_model.dart
│   └── api_response.dart
├── utils/                                # Shared utilities
│   ├── constants.dart
│   ├── extensions.dart
│   └── validators.dart
├── types/                                # Shared type definitions
│   ├── app_types.dart
│   └── api_types.dart
├── config/                                # Shared configuration
│   ├── app_config.dart
│   └── api_config.dart
└── ...
```

🧪 FRONTEND TESTING

Testing Structure (`/frontend/test/`)

```
frontend/test/
├── unit/
│   ├── features/
│   │   ├── inventory/
│   │   ├── sales/
│   │   ├── analytics/
│   │   └── loyalty/
│   ├── core/
│   │   ├── utils/
│   │   └── services/
│   └── widgets/
├── integration/
│   ├── api_integration_test.dart
│   ├── offline_sync_test.dart
│   └── barcode_scanner_test.dart
├── widget/
│   ├── inventory_widgets_test.dart
│   ├── sales_widgets_test.dart
│   └── loyalty_widgets_test.dart
├── e2e/
│   ├── app_test.dart
│   ├── offline_flow_test.dart
│   └── vendor_journey_test.dart
└── ...
```

📁 FRONTEND LOCAL STORAGE

Local Storage Structure (`/frontend/lib/core/storage/`)

```
frontend/lib/core/storage/
├── boxes/
│   ├── products_box.hive          # Product catalog
│   ├── inventory_box.hive         # Current stock levels
│   ├── sales_box.hive             # Offline sales queue
│   ├── customers_box.hive         # Local customer data
│   ├── loyalty_box.hive           # Points and rewards
│   └── sync_queue_box.hive        # Actions to sync
├── adapters/
│   ├── product_adapter.dart
│   ├── sale_adapter.dart
│   └── customer_adapter.dart
└── models/
    ├── local_product.dart
    ├── local_sale.dart
    └── local_customer.dart
```

🚀 DEPLOYMENT & DEVOPS

Frontend Configuration (`/frontend/`)

```
frontend/
├── pubspec.yaml                  # Dependencies and metadata
├── android/                     # Android configuration
├── ios/                         # iOS configuration (future)
├── assets/                      # Images, fonts, etc.
│   ├── images/
│   ├── icons/
│   └── fonts/
└── docs/                       # Frontend documentation
```

CI/CD Pipeline (`.github/workflows/`)

```
.github/
├── workflows/
│   ├── flutter-test.yml         # Run tests
│   ├── build-android.yml        # Build APK
│   ├── deploy-firebase.yml      # Deploy to Firebase
│   └── release-play-store.yml    # Publish to Play Store
└── actions/
    └── setup-flutter/
```

📄 DOCUMENTATION

Project Documentation (`/docs/`)

```
docs/
├── README.md                   # Project overview
├── SETUP.md                    # Development setup
├── DEPLOYMENT.md               # Deployment guide
├── API.md                      # API documentation
├── USER_GUIDE.md              # User manual
└── CONTRIBUTING.md             # Contribution guidelines
```

```

└─ ARCHITECTURE.md # Architecture decisions
...

---

## Cost Optimization

### Firebase Usage
- **Firestore**: Use caching, limit reads, optimize queries
- **Functions**: Use on-call triggers, optimize cold starts
- **Storage**: Compress images, use CDN
- **Hosting**: Static asset optimization

### Mobile Optimization
- **Bundle Size**: Code splitting, lazy loading
- **Memory**: Efficient data structures, dispose controllers
- **Battery**: Background sync optimization
- **Data**: Minimal sync payloads, compression

---

## Technology Stack Summary

### Mobile (Flutter)
- **Framework**: Flutter 3.x with Dart
- **State Management**: Provider/Riverpod
- **Local Storage**: Hive + SQLite
- **Barcode**: ML Kit + flutter_barcode_scanner
- **Voice**: speech_to_text package
- **Charts**: fl_chart for analytics
- **Networking**: dio with offline queue

### Backend (Firebase)
- **Database**: Firestore with offline persistence
- **Functions**: Cloud Functions (Node.js/TypeScript)
- **Auth**: Firebase Authentication
- **Storage**: Firebase Storage
- **Analytics**: Firebase Analytics
- **SMS**: Twilio integration

### Development Tools
- **Testing**: Flutter test, integration tests
- **CI/CD**: GitHub Actions
- **Distribution**: Google Play Console
- **Monitoring**: Firebase Crashlytics + Performance

---

## Key Features Implementation

### Offline-First Architecture
1. **Local First**: All operations write to local DB first
2. **Sync Queue**: Background sync when connectivity available
3. **Conflict Resolution**: Last-write-wins with timestamps
4. **Data Validation**: Local validation before sync

### Low-Tech User Experience
1. **Minimal Typing**: Voice input, barcode scanning
2. **Visual Interface**: Large buttons, clear icons
3. **Multilingual**: English + Shona support
4. **Offline Indicators**: Clear sync status display

### Cost-Effective Deployment
1. **Firebase Free Tier**: Start with free limits

```


2. **Progressive Scaling**: Pay-as-you-grow
3. **Android Focus**: Single platform optimization
4. **Lite Version**: Free tier with premium features

This structure provides a solid foundation for building VendorVault as an accessible, cost-effective solution for Zimbabwe's informal grocery vendors.